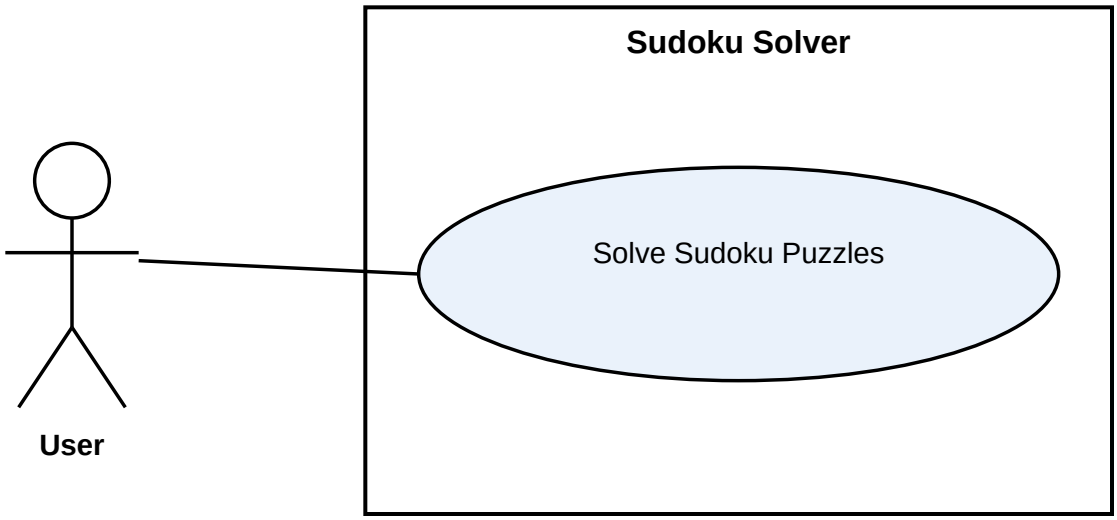
EECS 348 Assignment 4 - UML Use Case: Solve Sudoku Puzzles

Author: Andy Garcia. The use case was drafted with help from Claude (claude.ai) and reviewed against the program by the author.

UML Use Case Diagram



1. Identify / Discover the Use Case

Use case: Solve Sudoku Puzzles. **Actor:** User (the person who runs the program). **Goal of value to the actor:** get every solution, or a clear statement that none exists, for each of the five puzzle files.

2. Brief Description

This use case allows a User to run the program so that it reads five Sudoku puzzle files and finds every solution for each one. For each puzzle, it displays the file name, the original puzzle, and either all solutions found or a "No solution found" message.

3. Step-by-Step Flow (Outline)

1. Start program
2. Open puzzle file
3. Read and validate puzzle
4. Display original puzzle
5. Check given digits
6. Search for solutions
7. Report results

4. Detailed Description

1. Start Program

The User runs the program from the directory containing puzzle1.txt through puzzle5.txt. The system processes the files in order, 1 through 5.

2. Open Puzzle File

The system prints the name of the current puzzle file and tries to open it. If the file cannot be opened, the system prints "Error: Could not open file" and moves on to the next puzzle file.

3. Read and Validate Puzzle

The system reads 81 cells. Each cell must be a digit from 1 to 9 or an underscore (_) for a blank. If the file runs out of data or contains any other character, the system prints "Error: Invalid puzzle" for that file and moves on to the next puzzle file.

4. Display Original Puzzle

The system prints the original 9x9 grid, showing underscores for blank cells.

5. Check Given Digits

The system checks that no starting digit repeats in its row, its column, or its 3x3 block. If a digit repeats, the puzzle cannot have a solution, so the system skips the search and goes straight to reporting the result.

6. Search for Solutions

The system uses recursive depth-first search with backtracking. For each blank cell it tries the digits 1 through 9 that do not break a rule, moves to the next cell, and undoes the choice when it reaches a dead end. The search continues until every possibility has been tried, so all solutions are found. Each complete solution is printed as soon as it is found and is numbered in order.

7. Report Results

If no solution was found, the system prints "No solution found". Otherwise it prints "Solutions found: N" with the total count. The system then moves on to the next puzzle file. After puzzle5.txt, the program ends.